

Pintos Project 1: User Program (1)

담당 교수 : 김영재

조 / 조원 : 방지혁

개발 기간 : 2025.09.11 ~ 2025.10.08

1. 개발 목표

- 본 프로젝트는 pintos 내에서 user level program을 완전히 구현할 수 있도록 환경을 구축하는 것이다. 현재까지는 기본적인 kernel만 구현되어 있는 상황이다.
- 이를 위해서 첫번째 할 것은 argument passing을 구현하는 것이다. 명령어 인자들을 파싱하여 자료에 나와 있는 80x86 calling convention에 따라 stack을 구성한다. 두 번째는 user memory access 검증으로 잘못된 포인터, 메모리 접근, 커널 침범을 차단한다. 마지막으로 system call handler를 구축하여 user program에게 api를 제공한다. 또한, 추가적인 사용자 정의 시스템 콜도 구현한다.

2. 개발 범위 및 내용

A. 개발 범위

1. Argument Passing

유저 프로그램 실행에서 전달된 filename의 인자들을 파싱하고 스택에 쌓으며 저장되어 해당 인자들을 사용하게 된다.

2. User Memory Access

인자로 넘겨진 Pointer가 NULL이 아닌지, kernel이 아니라 PHYS_BASE 미만의 User address space인지, 페이지 디렉토리에 의해 매핑된 가상 메모리인지 여부를 확인한다. 만약 이를 위반할 시 위험할 수도 있기에 프로그램은 종료된다.

3. System Calls

halt, exit, exec, wait, read, write 같은 기본 system call을 구현했다. Exec, wait 구현을 위해 해당 로직을 process.c, thread.c, thread.h에 추가하였다. 또한, 사용자 정의 시스템콜 2개도 추가적으로 구현했다.

B. 개발 내용

- Argument Passing

- ✓ 커널 내 스택에 argument를 쌓는 과정 설명

process_execute 함수에서 start_process 함수를 호출하는데 이 함수는 load 함수를 호출한다. 인자로 전달받은 file_name 문자열에 대해 fn_copy에 복사한다. 이에 대해 만약 끝에 개행 문자가 있다면 이를 '\0'으로 바꿔준다. 공백 문자를 기준으로 토큰화해서 프로그램 이름을 argv[0]에, 나머지 인자들은 argv[1]부터 argv[argc-1]에 넣어준다. 그리고 setup_stack 함수를 통해 사용자 스택의 최상단 주소인 PHYS_BASE로 스택 포인터인 esp를 초기화한다. 앞서 언급한 분리된 문자열을 높은 주소부터 낮은 주소 방향으로 null-terminated string 형태로 저장한다. 이후 4바이트 경계에 맞춰 스택 포인터를 맞춰준다. 각 문자열의 주소를 다시 스택에 저장해준다. 이후 argv 리스트의 시작 주소, 인자 개수, 가짜 반환 주소를 순서대로 저장하면 완성이다.

- User Memory Access

- ✓ Pintos 상에서의 invalid memory access 개념을 간략히 설명

Pintos 상에서는 3가지 경우가 있다. 포인터 주소가 NULL이거나 주소가 PHYS_BASE 이상이어서 커널의 가상 주소 공간을 침범하거나 사용자 공간 내부에는 있지만 페이지 디렉토리에 매핑되어 있지 않은 경우이다. 이러한 접근이 있다면 kernel의 데이터 무결성을 훼손하거나 page fault로 이어져 kernel panic을 유발한다.

- ✓ Invalid memory access를 어떻게 막을 것인지 설명

Syscall.c에 check_address 함수를 통해 구현했다. 인자로 받은 vaddr에 대해 NULL인지 확인하고, is_user_vaddr() 함수를 사용하여 주소가 PHYS_BASE 미만인지 확인한다. 또한, pagedir_get_page 함수를 통해 주소가 현재 프로세스의 페이지 테이블에 매핑되어 있는지 여부를 확인한다. 만약 하나라도 위반할시 sys_exit(-1)을 호출하여 강제로 종료한다.

- System Calls

- ✓ 시스템 콜의 필요성에 대한 간략한 설명

만약 user-level program이 실행 도중 커널만이 수행할 수 있는 기능들을 수

행할 필요가 있을 때가 있다. 예를 들자면 I/O request 등이 있다. 이는 API 형태로 제공된다. 만약 이처럼 하지 않고 user program에 전적인 권한을 부여하게 된다면 시스템 보안에 위협이 될 수도 있다.

✓ 이번 프로젝트에서 개발할 시스템 콜에 대한 간략한 설명

- 1) **Exec**: 인자로 command-line을 받아 새로운 user program을 메모리에 로드하고 실행하는 시스템 콜이다.
 - 2) **Wait**: 시스템 프로그래밍 수업에서 배웠듯이 부모 프로세스가 자식 프로세스의 종료를 기다리고 그 종료 상태를 받아오는 system call이다.
 - 3) **Exit**: 현재 수행중인 프로세스를 종료하며 상태를 kernel에게 알리는 시스템콜이다.
 - 4) **Halt**: shutdown_power_off() 함수를 호출하여 system 전체를 종료시키는 system call이다.
 - 5) **Read**: 본 프로젝트에서는 open되어 있는 fd가 STDIN 뿐인데 해당 fd를 읽어 buffer에 저장한다.
 - 6) **Write**: 본 프로젝트에서 open되어 있는 fd인 STDOUT에 출력을 한다.
 - 7) **Fibonacci**: 넘겨받은 인자 1개에 대해 피보나치 수를 구해 계산하여 반환한다.
 - 8) **Max_of_four_int**: 넘겨 받은 인자 4개 중 가장 큰 값을 계산해 반환한다.
- ✓ 유저 레벨에서 시스템 콜 API를 호출한 이후 커널을 거쳐 다시 유저 레벨로 돌아올 때까지 각 요소를 설명

Additional.c 와 같은 사용자 프로그램에서 system call을 호출한다. Lib/user/syscall.c에 정의되어 있는 이 함수들은 스택에 인자들을 쌓고 인터럽트 0x30을 발생시키는 어셈블리 명령어를 실행한다. 해당 명령어는 기존의 레지스터 상태, 스택 포인터 등을 저장하며 유저 모드에서 커널모드로 CPU를 전환시키고 제어권을 커널로 넘긴다. 이후 우리가 만든 userprog/syscall.c의 syscall_handler 함수가 호출된다. 프로젝트에서 구현했듯 핸들러는 우선 주소의 유효성을 검증하고 스택에서 시스템 콜 번호를 읽어 해당하는 시스템 콜에 인자들을 넘겨준다. 해당 시스템 콜들은 작업을 수행한다. 결과를 eax에 저장하고 커널은 레지스터와 스택 포인터를 원래대로 되돌리며 다시 유저 모

드로 복귀해 다음 명령어로 돌아간다.

3. 추진 일정 및 개발 방법

A. 추진 일정

- 1주차 (9.11 ~ 9.17): 전체 계획 및 기본 환경 구축
 - ✓ process간 hierarchy 구축을 위해 스레드 구조체 수정
- 2주차 (9.18 ~ 9.24): argument passing을 위한 코드 작성
 - ✓ 스택 구성 코드 작성 및 정확한 파싱을 위한 process.c 내부 함수 변경
- 3주차 (9.25 ~ 10.01): system call 구현 및 추가 시스템 콜 작성
 - ✓ 주소 유효성 확인 함수 작성, system call 함수 작성, process_wait, process_exit, thread_init, thread_schedule_tail 함수 수정
- 4주차 (10.02 ~ 10.08): 디버깅 및 보고서 작성
 - ✓ Make check를 하며 잘못 구현된 부분에 대한 코드 수정

B. 개발 방법

1) Argument Passing 구현

1. Load 함수 수정

Strtok_r함수로 공백을 기준으로 토큰화해서 char *argv[128] 배열에 각 인자들의 포인터를 저장해준다. Argc에 인자 개수를 넣어준다.

2. 스택 구성 코드 작성

인자들을 역순으로 Push한다. Word-align을 위한 패딩을 추가하고 NULL pointer도 추가하고 인자들의 주소를 다시 스택에 저장한다. 마지막으로 argc, argv, fake return address도 저장한다.

2) User Memory accesss 구현

1. Address valid check 함수 추가

Syscall.c에 추가하여 NULL 여부를 확인하고 is_user_vaddr()로 유저영역인지 체크하고 마지막으로 pagedir_get_page 함수로 매핑여부를

확인하여 위반할시 프로세스를 종료한다.

3) System Call 구현

1. Syscall_handler 함수 구현

f->esp의 주소도 확인하고, 유효하다며 시스템 콜 번호를 확인하여 그 번호에 맞는 시스템콜 함수를 호출하여 주소를 체크하여 인자를 넘겨주고 반환값은 f->eax에 저장한다.

2. Process_execute 함수 수정

Command line을 strtok_r함수를 사용하여 파싱해서 가장 첫번째 토큰인 프로그램 이름을 스레드 이름으로 사용한다. 자식 프로세스의 부모 포인터를 설정해주고 child_list에 추가해준다. Sema_down 함수를 사용하여 자식이 로드 완료때까지 대기하도록 설정한다.

3. Start_process 함수 수정

Load 함수를 호출하고 나서 load_success를 설정해준다. 이후 sema_up으로 부모 프로세스에게 Load 성공을 알린다.

4. System call 함수 구현

Sys_halt()의 경우 shut_down_power_off() 함수를 호출하여 시스템을 종료시킨다. Sys_exit()는 종료 메시지를 출력하고 exit_status를 저장하여. Thread_exit을 호출한다. Sys_exec는 process_execute를 호출하여 새로운 프로세스를 실행시킨다. Sys_wait은 process_wait을 호출하여 자식 프로세스를 대기한다. Sys_read는 STDIN 즉 fd가 0인 기본적인 경우로 input_getc()를 사용하여 입력을 받도록 구현했다. 이외의 경우에는 -1을 반환한다. Sys_write도 마찬가지로 fd가 1인 기본적인 경우로 putbuf()를 사용하여 출력하도록 구현했다. 또한 추가적인 시스템 콜인 fibonacci와 max_of_four_int도 구현했다.

5. Process간 hierarchy 구축 및 상태 관리를 위한 thread 구조체 수정

Thread 구조체 필드 추가 변수

struct thread *parent: 부모 프로세스를 가리키는 포인터

struct list child_list: 자식 프로세스 리스트

struct list_elem child_elem: 자식 프로세스를 리스트에 넣기 위한 요소

int exit_status: 프로세스 종료 상태 저장

bool load_success: 프로세스 로드 성공 여부

bool waited: 이미 wait된 자식인지 여부

struct semaphore exit_sema: 부모가 자식의 종료를 기다리기 위한 세마포어

struct semaphore load_sema: 자식 프로세스 로드를 기다리기 위한 세마포어

6. process 간 hierarchy 구축 및 상태 관리를 위한 init_thread 함수 수정

list_init(&(t->child_list)): 자식 리스트 초기화

sema_init(&(t->exit_sema), 0): exit 세마포어 초기화

sema_init(&(t->load_sema), 0): load 세마포어 초기화

exit_status = -1, load_success = false, waited = false

7. thread_schedule_tail 함수 수정

종료된 스레드의 경우 부모 프로세스가 있으면 부모가 exit_status 읽도록 자식 스레드 구조체 유지

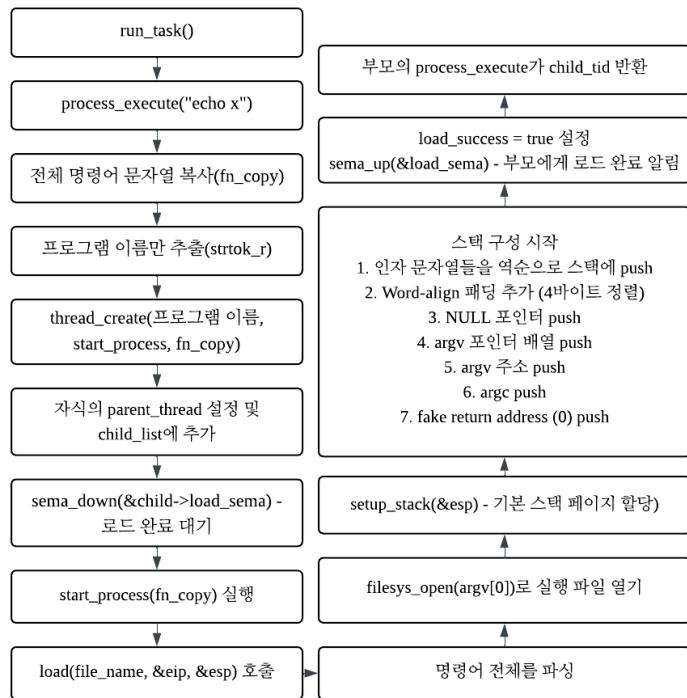
8. process_wait 함수 수정

child_list를 순회하여 해당 tid의 자식을 찾는데 유효하지 않은 tid이거나 이미 wait된 자식이면 -1을 반환한다. 이후 waited 플래그를 true로 설정하여 중복으로 wait 되는 것을 방지하고, sema_down(&child->exit_sema)로 자식이 종료하기 전까지 wait한다. 마지막으로 자식의 exit_status를 반환하고 child_list에서 제거하고 나서야 메모리 해제가 이루어진다.

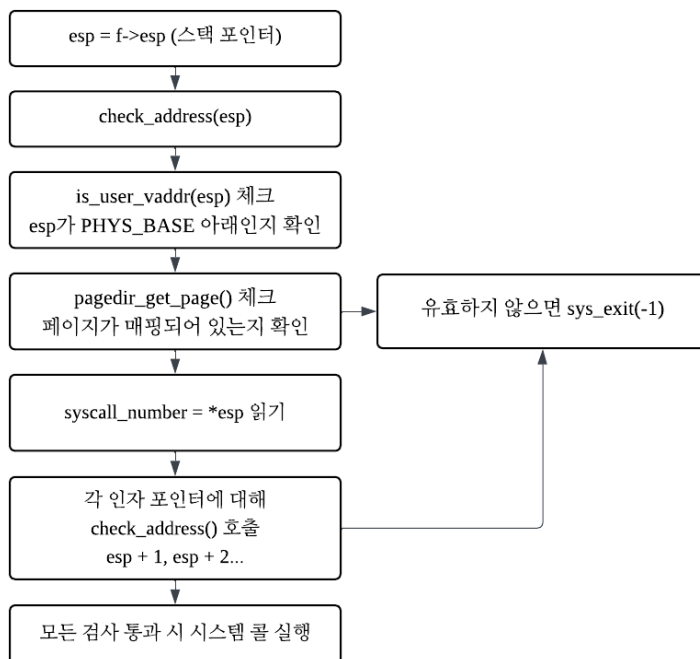
4. 연구 결과

A. Flow Chart

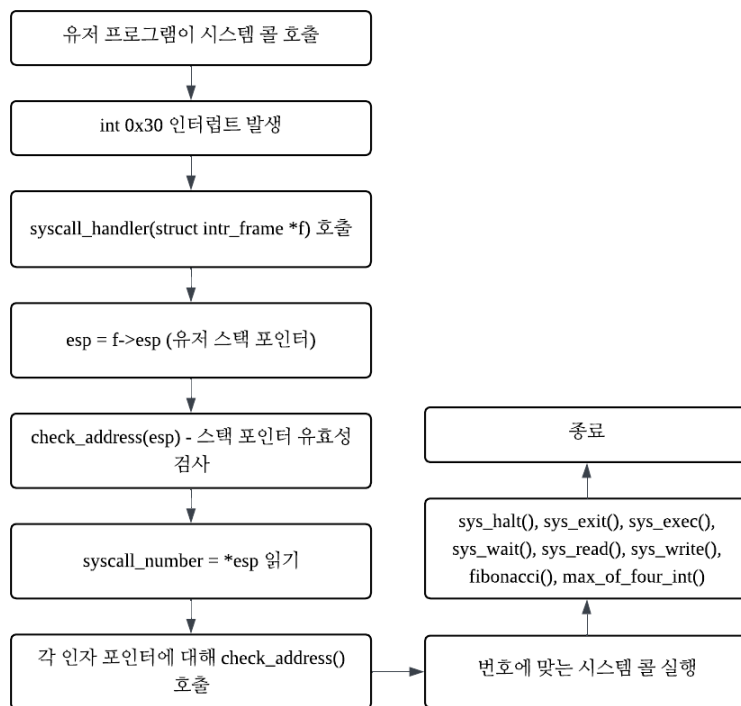
1. Argument Passing



2. User Memory Access



3. System Calls



다음장에 계속

B. 제작 내용

1. Argument Passing

src/userprog/process.c

load 함수: 해당 함수는 file_name으로 주어진 프로그램을 로드합니다.

```
bool load(const char *file_name, void (**eip)(void), void **esp) {
    struct thread *t = thread_current();
    struct Elf32_Ehdr ehdr;
    struct file *file = NULL;
    off_t file_ofs;
    bool success = false;
    int i;
    char *argv[128]; // argument list
    int argc = 0;    // argument count
    char *token, *temp_ptr;

    /* Allocate and activate page directory. */
    t->pagedir = pagedir_create();
    if (t->pagedir == NULL) goto done;
    process_activate();

    /* Parse the command line into arguments */
    char *fn_copy = pallocc_get_page(0);
    if (fn_copy == NULL) return false;
    strcpy(fn_copy, file_name, PGSIZE);

    int len = strlen(fn_copy);
    if (len > 0 && fn_copy[len - 1] == '\n') {
        fn_copy[len - 1] = '\0';
    }

    token = strtok_r(fn_copy, " ", &temp_ptr);
    while (token != NULL && argc < 128) {
        argv[argc] = token;
        argc++;
        token = strtok_r(NULL, " ", &temp_ptr);
    }
    /*
    printf("DEBUG: parsing result: argc=%d\n", argc);
    for (int i = 0; i < argc; i++) {
        printf("DEBUG: argv[%d]='%s'\n", i, argv[i]);
    }
    */
    /* Open executable file. */
    file = filesys_open(argv[0]);
    if (file == NULL) {
        printf("load: %s: open failed\n", argv[0]);
        goto done;
    }
}
```

File_name은 "echo x y"로 사용자가 입력한 전체 문자열이 들어옵니다. 이를 파싱해줘야 하기 때문에 기존 코드에서 추가적으로 char *argv[128]과 argc, char *token, temp_ptr을 선언해줍니다. 이후 기존 코드에서는 새 프로세스를 위한 페이지 디렉토리를 pagedir_create()함수를 사용하여 생성하고, process_activate()함수를 사용하여 현재 thread의 page director를 CPU에 로드해줍니다.

이후부터는 본인이 작성한 코드인데 pallocc_get_page(0)을 호출해 file_name을 복사해준다.

이후 복사본의 문자열끝에 개행문자가 있다면 이를 제거해준다. 이후 복사받은 명령줄을 공백 기준으로 토큰화해줄 것이다. Strtok_r함수를 사용해 fn_copy를 공백을 기준으로 분리해주는데, 공백은 \0으로 바꿔준다. 각 토큰들은 argv 배열에 저장해주고 인자 개수를 세는 argc변수를 증가시켜준다. Argv[0]은 프로그램에 이름이 들어가있을 것이기에 filesys_open()함수에 인자로 이전처럼 명령어 문자열 전체인 file_name이 아니라 argv[0]을 넣어준다.

```

if (!setup_stack(esp)) goto done;

/* 인자 넣기 */
char *arg_addresses[128];
for (i = argc - 1; i >= 0; i--) {
    size_t len = strlen(argv[i]) + 1;
    *esp -= len;
    strcpy(*esp, argv[i], len);
    arg_addresses[i] = *esp;
}

/* word align */
while ((uint8_t)*esp % 4 != 0) {
    *esp = (uint8_t)*esp - 1;
    *((uint8_t *)*esp) = 0;
}

/* null */
*esp -= 4;
*((char **)*esp) = NULL;

for (i = argc - 1; i >= 0; i--) {
    *esp -= 4;
    *((char **)*esp) = arg_addresses[i];
}

/* argv */
char **argv_addr = *esp;
*esp -= 4;
*((char **)*esp) = argv_addr;

/* argc */
*esp -= 4;
*((int *)*esp) = argc;

/* fake return address */
*esp -= 4;
*((void **)*esp) = NULL;

// printf("hex dump in construct_stack start\n\n");
// hex_dump(*esp, *esp, 100, true);
/* Start address. */
*eip = (void (*)(void))ehdr.e_entry;

success = true;
one:
/* We arrive here whether the load is successful or not. */
file_close(file);
palloc_free_page(fn_copy);
return success;

```

이후 setup_stack(esp)함수를 통해 4KB짜리 스택 페이지를 할당한다. *esp는 현재 PHYS_BASE(0xc0000000)를 가리키고 있을 것이다. 즉 맨 위이며 스택은 아래방향으로 자란다. 이후 우리는 인자를 역순으로 push 해야 한다. 인자 문자열들을 역순으로 push 하는데, 공간의 크기는 'W0'을 포함해야 하므로, strlen(인자) + 1만큼 스택 포인터를 빼준다. 이후 strcpy 함수를 사용한다. 인자로 들어간 size - 1개의 문자만 복사하고 마지막에 NULL 문자를 추가해주기 때문에 strcpy함수를 사용하였다. 이후 해당 주소들을 다시 스택에 넣어야 하기 때문에 *esp 현재 위치 값을 arg_addresses배열에 넣어준다. 이후 4바이트 정렬이 필요한데 반복문을 돌아야한다. 사실 esp는 void v 포인터의 포인터로 스택 포인터의 주소를 담은 포인터이다. 스택 포인터 값을 얻으려면 역참조를 해야 한다. 그리고 void 포인터의 포인터이기에 *esp에 산술 연산을 하기 위해

서 (uint8_t)로 캐스팅해준다. *esp가 4로 나누어지는지 확인하며 1바이트씩 감소시키며 가리키는 위치에 0을 1바이트만큼 써주는 것이다. 이후 argv배열의 종료를 알려야 하기 때문에 NULL을 넣어야한다. NULL은 포인터 값으로 4바이트 크기이기에 스택 포인터를 4바이트 감소 시키고 해당 위치를 char **로 캐스팅해서 NULL을 저장한다. Char **로 캐스팅하는 이유는 4바이트짜리 공간으로 해석해야 하기 때문이다. 마찬가지로 char*을 넣어야 하기 때문에 char**로 캐스팅을 해주고 스택포인터를 4바이트씩 감소시키며 arg_addresses를 역순으로 집어 넣는다. 이 와중 argv배열의 시작 주소를 저장해야 하기 때문에 반복문이 끝나고 나서의 *esp 값을 char **argv_addr 변수에 넣고 4바이트만큼 스택을 감소시키고 해당 값을 넣는다. 이후 argc도 push하

기 위해서 4바이트 감소시키고 int*로 캐스팅하고 역참고해서 값을 넣고 fake return address 도 push 한다.

2. User Memory Access

```
void check_address(void *vaddr) {
    if (vaddr == NULL || !is_user_vaddr(vaddr) || pagedir_get_page(thread_current()->pagedir, vaddr)
        // printf("Invalid address: %p\n", vaddr);
        sys_exit(-1);
    }
}
```

Check_address 함수에서 우선 NULL 포인터인지 체크하고 is_user_vaddr()는 인자로 들어온 주소가 PHYS_BASE이상인지(커널영역)인지 체크한다. 이는 src/threads/vaddr.h에 포함되어 있다. 또한 Pagedir_get_page()함수를 통해 주소가 실제 매핑된 페이지인지 체크한다. 이는 userprog/pagedir.c에 구현되어 있는 함수로 가상 주소가 물리 메모리에 매핑되어 있는지 확인하는 함수로 매핑되어 있다면 물리주소를 반환하고 아닐시 NULL을 반환한다. 인자로 현재 프로세스의 페이지 테이블인 thread_current()->pagedir와 검사할 가상 주소인 vaddr를 넣어 매핑 여부를 확인한다. 아무리 유저영역이여도 스택 혹은 힙 영역이 아닌 곳에 접근하면 안되기 때문이다.

3. System Calls

```
static void syscall_handler(struct intr_frame *f) {
    uint32_t *esp = f->esp;
    check_address(esp);
    int syscall_number = *esp;
```

우선 유저 스택 포인터를 가져오고 2번 항목에서 서술한 check_address 함수로 해당 주소의 유효성을 확인한다. 그리고 역참조 해서 시스템 콜 번호에 따라 분기하며 시스템 콜 함수를 수행한다.

위의 첨부된 코드와 같이 말이다.

```
switch (syscall_number) {
    case SYS_HALT:
        sys_halt();
        break;
}

void sys_halt(void) { shutdown_power_off(); }
```

sys_halt의 경우 간단한데 인자가 따로 없기 때문에 따로 체크할 인자는 없고 devices/shutdown.c에 정의되어 있는 shutdown_power_off()를 호출하고 시스템을 종료한다.

```
case SYS_EXIT:
    check_address(esp + 1);
    sys_exit((int)(*(esp + 1)));
    break;

void sys_exit(int status) {
    struct thread *cur = thread_current();
    printf("%s: exit(%d)\n", thread_name(), status);
    cur->exit_status = status;
    thread_exit();
}
```

sys_exit는 정수형 인자 하나를 받는다. 인자인 esp + 1을 check_address()로 검증한 후, 해당 주소에서 종료 상태 값을 읽는다. 현재 스레드의 이름과 종료 상태를 printf()로 출력하고, exit_status에 상태 값을 저장한 뒤 thread_exit()를 호출하여 프로세스를 종료한다. Thread_exit함수에서 부모 프로세스가 있다면 exit_sema를 up하여 부모에게 종료를 알린다. 이 부분에 대해서는 추후 다시 설명하겠다. 참고로 종료상태 저장을 위해 thread 구조체에 exit_status 변수를 추가한다.

```
struct thread {
    /* Owned by thread.c. */
    tid_t tid; /* Thread identifier. */
    enum thread_status status; /* Thread state. */
    char name[16]; /* Name (for debugging purposes). */
    uint8_t *stack; /* Saved stack pointer. */
    int priority; /* Priority. */
    struct list_elem allelem; /* List element for all threads list. */

    /* Shared between thread.c and synch.c. */
    struct list_elem elem; /* List element. */

#ifdef USERPROG
    /* Owned by userprog/process.c. */
    uint32_t *pagedir; /* Page directory. */
    struct thread *parent_thread; /* 자식에서 부모로 이동하기 위한 ptr */
    struct list_elem child_elem; /* 자식이 부모의 리스트에 연결될 때 사용하는 e */
    struct list child_list; /* 자식 리스트 */
    int exit_status;
    bool load_success;
    bool waited;
    struct semaphore exit_sema;
    struct semaphore load_sema;
#endif
};
```

```

case SYS_EXEC:
    check_address(esp + 1);
    const char *cmd_line = (const char *)*(esp + 1);
    check_address(cmd_line);
    f->eax = sys_exec((const char *)*(esp + 1));
    break;
}

int sys_exec(const char *cmd_line) {
    return process_execute(cmd_line);
}

```

sys_exec는 문자열 포인터 인자 하나를 받는다. esp + 1 위치의 주소를 검증한 후, 또 다시 해당 주소에서 문자열 포인터를 읽어와서 check_address()로 추가 검증한다. Malicious한 공격이 있을 수 있기 때문이다. 검증이 완료되면 process_execute()를 호출하여 새로운 프로세스를 생성하여 반환된 process id를 f->eax에 저장하여 반환한다. 만약 실패하면 -1을 반환한다. 이 부분도 추후 설명하겠다.

```

case SYS_WAIT:
    check_address(esp + 1);
    f->eax = sys_wait((int)*(esp + 1));
    break;
}

int sys_wait(int pid) { return process_wait(pid); }

```

sys_wait는 정수형 인자 하나를 받는다. esp + 1 주소를 검증한 후, 대기할 자식 프로세스의 PID를 읽는다. process_wait()를 호출하여 해당 프로세스가 종료될 때까지 대기하고, 자식의 종료 상태를 받아 f->eax에 저장한다. 유효하지 않거나 이미 wait한 자식이면 -1을 반환한다. 이 부분도 추후 설명하겠다.

```

case SYS_READ:
    check_address(esp + 1);
    check_address(esp + 2);
    check_address(esp + 3);
    f->eax = sys_read((int)*(esp + 1), (const void *)*(esp + 2), (unsigned)*(esp + 3));
    break;
}

```

```

int sys_read(int fd, void *buffer, unsigned size) {
    unsigned cnt = 0;
    char *buf = (char *)buffer;

    // fd가 0인 경우 (표준 입력) ==> input_getc() 사용
    if (fd == 0) {
        while (cnt < size) {
            char c = input_getc();
            buf[cnt++] = c;
            // if (c == '\n') break;
        }
        // buf[cnt] = '\0';
        return cnt;
    }
    // 다른 fd의 경우 file_read() 사용 ==> proj1에서는 미구현
    else return -1;
}

```

sys_read는 세 개의 인자를 받는다. 각 인자의 주소를 각각 검증한 후, fd(파일 디스크립터), buffer(버퍼 주소), size(크기)를 읽어온다. fd가 0(STDIN)인 경우 input_getc()를 사용하여 키보드 입력을 인자로 들어온 size만큼 읽어 buffer에 저장하고, 읽은 바이트 수를 반환한다. fd가 0이 아닌 경우(파일 읽기)는 Project 1에서 미구현하여 -1을 반

환하도록 한다.

```

case SYS_WRITE:
    check_address(esp + 1);
    check_address(esp + 2);
    check_address(esp + 3);
    f->eax = sys_write((int)(*(esp + 1)), (const void *)(*(esp + 2)), (unsigned)(*(esp + 3)));
    break;

```

```

int sys_write(int fd, const void *buffer, unsigned size) {
    // check_address(buffer);
    if (fd == 1) {
        // printf("SYS_WRITE buffer content: %.*s\n", size, (char *)buffer);
        putbuf(buffer, size);
        return size;
    }
    return -1;
}

```

sys_write는 마찬가지로 세 개의 인자를 받는다. 각 인자의 주소를 각각 검증한 후, fd(파일 디스크립터), buffer(버퍼 주소), size(크기)를 읽어온다. fd가 1(STDOUT)인 경우 lib/kernel/console.c에 정의된 putbuf()를 사용하여 buffer의 내용을 size만큼 console에 출력하고, 출력한 바이트 수를 반환한다. fd가 1이 아닌 경우(파일 쓰기)는 Project 1에서 미구현하여 -1을 반환하도록 한다.

이제 wait, exec에 대한 추가적인 설명이다.

```

#ifdef USERPROG
    /* Owned by userprog/process.c. */
    uint32_t *pagedir; /* Page directory. */
    struct thread *parent_thread; /* 자식에서 부모로 이동하기 위한 ptr */
    struct list_elem child_elem; /* 자식이 부모의 리스트에 연결될 때 사용하는 elem */
    struct list child_list; /* 자식 리스트 */
    int exit_status;
    bool load_success;
    bool waited;
    struct semaphore exit_sema;
    struct semaphore load_sema;
#endif

```

해당 필드들은 USERPROG 블록 안에 있다. 이 중 추가한 변수는 struct thread *parent_thread로 부모 프로세스를 가리키는 포인터이다. process_execute()에서 자식 프로세스를 생성할 때 설정되며, 자식이 종료될 때 부모에게 알리거나 부모가 자식을 wait할 때 사용된다.

또한, struct list_elem child_elem는 자식 프로세스가 부모의 child_list에 연결될 때 사용하는 리스트 요소이다. 부모 프로세스는 이 요소를 통해 모든 자식들을 리스트로 관리할 수 있다. list_push_back()으로 부모의 자식 리스트에 추가되고, list_remove()로 제거된다.

또한, `struct list child_list`는 모든 자식 프로세스들을 관리하는 리스트이다. `process_wait()`에서 기다려야 할 특정 자식을 찾을 때 이 리스트를 순회한다. `list_init()`으로 초기화되며, 앞서 언급한 각 자식 프로세스의 `child_elem`을 통해 연결된다.

`int exit_status`는 언급했기에 생략하겠다.

`bool load_success`는 자식 프로세스의 실행 파일 로드가 성공했는지 여부를 나타내는 Boolean 플래그이다. `load()` 함수에서 설정되며, 부모 프로세스가 `process_execute()`에서 확인하여 자식 프로세스 생성 성공여부를 확인할 수 있다.

`bool waited`는 이미 `wait`이 호출되었는지 확인하는 플래그이다. 중복 `wait`을 방지하기 위해 사용된다. `Process_wait()`에서 `true`로 설정된다.

`struct semaphore exit_sema`는 부모 프로세스가 자식의 종료를 기다릴 때 사용하는 세마포어이다. 0이 초기값이며, 자식이 `thread_exit()`에서 `sema_up(&exit_sema)` 함수를 호출하여 부모에게 종료를 알린다. 부모는 `process_wait()`에서 `sema_down(&child->exit_sema)`를 호출하여 자식이 종료될 때까지 `blocked` 상태로 대기하게 되는 것이다.

마지막으로 `struct semaphore load_sema`가 있다. 부모 프로세스가 자식의 실행 파일 로드 완료로 기다릴 때 사용하는 세마포어로 마찬가지로 초기값이 0이다. `Load()` 함수 완료 후 자식 프로세스에서 `sema_up`을 호출하여 부모에게 로드 완료를 알리는 것이다. 이후 부모는 기다리다가 `Load_success` 플래그로 성공 여부를 판단한다.

다음장에 이어서


```

tid_t process_execute(const char *file_name) {
    char *fn_copy;
    char *fn_copy2;
    char *temp_ptr;
    char *cmd_name;
    tid_t tid;

    /* race condition 막기 위해 우선 파일 이름 복사 */
    fn_copy = pallocc_get_page(0);
    if (fn_copy == NULL) return TID_ERROR;
    strcpy(fn_copy, file_name, PGSIZE);

    /* 파일 이름만 추출 (첫 번째 공백 전까지) */

    fn_copy2 = pallocc_get_page(0);
    if (fn_copy2 == NULL) {
        pallocc_free_page(fn_copy);
        return TID_ERROR;
    }
    strcpy(fn_copy2, file_name, PGSIZE);
    cmd_name = strtok_r(fn_copy2, " ", &temp_ptr);
    if (cmd_name == NULL) {
        pallocc_free_page(fn_copy);
        pallocc_free_page(fn_copy2);
        return TID_ERROR;
    }

    /* Create a new thread to execute FILE_NAME. */
    tid = thread_create(cmd_name, PRI_DEFAULT, start_process, fn_copy);
}

```

Process_execute 함수를 수정했다. 이는 부모 측 함수라고 볼 수 있다. Race condition 을 방지하기 위해서 파일이름을 fn_copy에 복사해주는데 이는 start_process에 쓰인다. 기존과 달리 또 다른 복사본인 fn_copy2를 만들어 첫번째 프로그램 이름을 cmd_name에 저장한다. 이는 이후 thread_create를 호출할 때 스레드 이름을 설정하는데 사용된다. 다음은 자식 설정 부분이다. 다음 장에 이어서

```

static void
init_thread (struct thread *t, const char *name, int priority)
{
    enum intr_level old_level;

    ASSERT (t != NULL);
    ASSERT (PRI_MIN <= priority && priority <= PRI_MAX);
    ASSERT (name != NULL);

    memset (t, 0, sizeof *t);
    t->status = THREAD_BLOCKED;
    strncpy (t->name, name, sizeof t->name);
    t->stack = (uint8_t *) t + PGSIZE;
    t->priority = priority;
    t->magic = THREAD_MAGIC;

    old_level = intr_disable ();
    list_push_back (&all_list, &t->allelem);
    intr_set_level (old_level);

    list_init(&(t->child_list));
    t->parent_thread = NULL;
    t->exit_status = -1;
    t->load_success = false;
    t->waited = false;
    sema_init(&(t->exit_sema), 0);
    sema_init(&(t->load_sema), 0);
}

```

init_thread는 thread_create 함수 내부에서 호출되는데 수정한 부분에 대해 설명하자면 t->child_list를 빈 리스트로 초기화하고 부모 thread는 NULL로, exit_status는 기본값인 -1로 설정한다. load_success와 waited 플래그는 모두 false로 초기화한다. 또한, 자식 종료 대기 위한 세마포어인 exit_sema, 로딩 완료 대기 위한 세마포어인 load_sema 또한, 0으로 초기화해준다.

```

if (tid != TID_ERROR) {
    struct thread *child = get_thread_by_tid(tid);
    if (child != NULL) {
        struct thread *cur = thread_current();
        child->parent_thread = cur;
        list_push_back(&(cur->child_list), &(child->child_elem));

        // 자식이 로드 완료될 때까지 대기
        sema_down(&child->load_sema);

        // 로드 실패하면 -1 리턴
        if (!child->load_success) {
            list_remove(&(child->child_elem));
            tid = TID_ERROR;
        }
    }
}
palloc_free_page(fn_copy2);
// if (tid == TID_ERROR) palloc_free_page(fn_copy);
return tid;

```

이후 생성된 tid를 통해 자식 스레드의 구조체 child를 가져온다. 부모와 자식을 연결해주는 과정인데, 자식 스레드의 parent_thread를 현재 스레드로 설정한다. 그리고 list_push_back 함수를 통해 현재 스레드의 자식 목록에 자식을 추가한다. 그리고 sema_down 함수를 통해 자식이 로딩을 완료 하고 load_sema를 up할 때까지 대기시킨다. 대기에서 풀려나면 load_success 플래그를 확인하고 load 실패 시 자식 목록에서 제거하고 tid값을 TID_ERROR로 설정한다. 이후 프로그램 이름을 추출하는 데 사용했던 임시 페이지 fn_copy2를 해제한다.

```

struct thread *get_thread_by_tid(tid_t tid) {
    for (struct list_elem *e = list_begin(&all_list); e != list_end(&all_list); e = list_next(e)) {
        struct thread *t = list_entry(e, struct thread, allelem);
        if (t->tid == tid) {
            return t;
        }
    }
    return NULL;
}

```

여기서 쓰인 get_thread_by_tid는 주어진 스레드 id 즉 tid로 일치하는 thread를 모든 스레드 리스트를 순회하며 찾는다. 그리고 찾으면 해당 스레드 구조체의 포인터를 즉시 반환하고 종료한다.

다음 장으로

```

static void start_process(void *file_name_) {
    char *file_name = file_name_;
    struct intr_frame if_;
    bool success;

    /* Initialize interrupt frame and load executable. */
    memset(&if_, 0, sizeof if_);
    if_.gs = if_.fs = if_.es = if_.ds = if_.ss = SEL_UDSEG;
    if_.cs = SEL_UCSEG;
    if_.eflags = FLAG_IF | FLAG_MBS;
    success = load(file_name, &if_.eip, &if_.esp);
    thread_current()->load_success = success;
    sema_up(&thread_current()->load_sema); // 부모에게 로드 완료 알림
    /* If load failed, quit. */
    pallocc_free_page(file_name);
    if (!success) thread_exit();

    /* Start the user process by simulating a return from an
       interrupt, implemented by intr_exit (in
       threads/intr-stubs.S). Because intr_exit takes all of its
       arguments on the stack in the form of a 'struct intr_frame',
       we just point the stack pointer (%esp) to our stack frame
       and jump to it. */
    asm volatile("movl %0, %%esp; jmp intr_exit" : : "g"(&if_) : "memory");
    NOT_REACHED();
}

```

Start_process 함수를 수정했다. 이는 자식 측 함수라고 볼 수 있다. 수정한 부분에 대해 설명하자면 부모에게 sema_up(&thread_current()->load_sema)를 통해 로딩 성공을 알리는 것이다. 이후 file_name을 free 해준다. 이후 커널모드에서 유저 모드로 전환하고 eip에는 main 함수 주소, esp에는 우리가 만든 스택이 들어가는 것이다. 이후 main함수에서 return 0;을 하면 종료하며 커널모드로 전환하며 exit 시스템 콜을 호출하게 되는 것이다. Sys_Exit()을 하면서 thread_exit()을 호출하게 되는데

```

void
thread_exit (void)
{
    ASSERT (!intr_context ());

#ifdef USERPROG
    struct thread *cur = thread_current();
    if (cur->parent_thread != NULL) {
        sema_up(&cur->exit_sema);
    }
    process_exit ();
#endif

    /* Remove thread from all threads list, set our status to dying,
       and schedule another process. That process will destroy us
       when it calls thread_schedule_tail(). */
    intr_disable ();
    list_remove (&thread_current()->allelem);
    thread_current ()->status = THREAD_DYING;
    schedule ();
    NOT_REACHED ();
}

```

이 thread_exit 함수에서 부모가 있으면 exit_sema를 올리며 wait하고 있는 부모를 깨운다. 그리고 Process_exit을 호출하며 리소스를 해제한다.

그리고 process_wait에 대해 설명하겠다.

```
int process_wait(tid_t child_tid UNUSED) {
    struct thread *cur = thread_current();
    struct thread *child = NULL;

    struct list_elem *e;
    for (e = list_begin(&(cur->child_list)); e != list_end(&(cur->child_list)); e = list_next(e)) {
        struct thread *t = list_entry(e, struct thread, child_elem);
        if (t->tid == child_tid) {
            child = t;
            break;
        }
    }
    if (child == NULL) return -1; // 유효하지 않은 tid

    if (child->waited) {
        return -1;
    }
    child->waited = true;
    sema_down(&child->exit_sema); // 자식이 종료될 때까지 대기
    int status = child->exit_status;
    list_remove(&child->child_elem); // 부모의 자식 리스트에서 제거
    palloc_free_page(child);
    return status;
}
```

수정한 부분에 대해 설명하자면 현재 스레드가 관리하는 자식 스레드 목록을 순회한다. 인자로 받은 tid와 일치하는 스레드를 찾고 없거나 이미 기다려졌으면 -1을 반환하며 종료한다. 그리고 sema_down을 통해 자식이 종료될때까지 대기한다. 앞서 말했듯 자식은 thread_exit 함수에서 sema_up을 호출하여 자신이 종료되었음을 알린다. 그리고 종료 status를 회수하고 자식 리스트에서 제거한 뒤 메모리를 해제해준다.

다음장에 이어서

```

void
thread_schedule_tail (struct thread *prev)
{
    struct thread *cur = running_thread ();

    ASSERT (intr_get_level () == INTR_OFF);

    /* Mark us as running. */
    cur->status = THREAD_RUNNING;

    /* Start new time slice. */
    thread_ticks = 0;

#ifdef USERPROG
    /* Activate the new address space. */
    process_activate ();
#endif

    /* If the thread we switched from is dying, destroy its struct
       thread.  This must happen late so that thread_exit() doesn't
       pull out the rug under itself.  (We don't free
       initial_thread because its memory was not obtained via
       pallocc().) */
    if (prev != NULL && prev->status == THREAD_DYING && prev != initial_thread)
    {
        ASSERT (prev != cur);
        if (prev->parent_thread != NULL) {
            return;
        }
        pallocc_free_page(prev);
    }
}

```

문맥전환 시 현재 스레드를 활성화하고 이전 스레드가 종료 상태라면 리소스를 해제하는 함수이다. 여기서 수정한 점은 종료된 스레드가 부모 스레드가 있을 시, 부모가 나중에 process_wait을 통해 메모리를 해제하도록 return을 하는 것이다.

4. Additional System calls

```
case SYS_FIBONACCI:
    check_address(esp + 1);
    f->eax = fibonacci((int)*(esp + 1));
    break;
case SYS_MAX_OF_FOUR_INT:
    check_address(esp + 1);
    check_address(esp + 2);
    check_address(esp + 3);
    check_address(esp + 4);
    f->eax = max_of_four_int((int)*(esp + 1), (int)*(esp + 2), (int)*(esp + 3), (int)*(esp + 4));
    break;
```

```
int fibonacci(int n) {
    if (n < 0) return 0;
    if (n <= 1) return n;
    int x0 = 0, x1 = 1;
    int res = 1;
    for (int i = 2; i <= n-1; i++) {
        x0 = x1;
        x1 = res;
        res = x0 + x1;
    }
    return res;
}

int max_of_four_int(int a, int b, int c, int d) {
    int max = a;
    if (b > max) max = b;
    if (c > max) max = c;
    if (d > max) max = d;
    return max;
}
```

마찬가지로 인자에 대해 valid한지 체크한다.

Fibonacci 함수는 반복문 형식으로 구현했다. N이 0보다 작으면 0을 반환하고 1보다 작거나 같으면 n을 반환한다. 2이상부터 n까지 반복하며 이전 두 항을 더하여 새로운 항을 계산하도록 코드를 작성했다.

Max_of_four_int 함수는 간단하게 작성했다. 그냥 순차적으로 비교를 했다 최댓값을 업데이트한다.

```

#ifndef USERPROG_SYSCALL_H
#define USERPROG_SYSCALL_H

void syscall_init (void);
void check_address(void *vaddr);
void sys_halt(void);
void sys_exit (int status);
int sys_exec(const char *cmd_line);
int sys_wait(int pid);
int sys_read(int fd, void *buffer, unsigned size);
int sys_write(int fd, const void *buffer, unsigned size);
int fibonacci(int n);
int max_of_four_int(int a, int b, int c, int d);

#endif /* userprog/syscall.h */

```

Syscall.h헤더에 함수 선언도 추가했으며

Lib/user/syscall.h에도 userlevel에서 해당 함수 선언들을 추가했다.

Lib/user/syscall.c를 확인하면

```

#define syscall4(NUMBER, ARG0, ARG1, ARG2, ARG3) \
({ \
    int retval; \
    asm volatile( \
        "pushl %[arg3]; pushl %[arg2]; pushl %[arg1]; pushl %[arg0]; " \
        "pushl %[number]; int $0x30; addl $20, %%esp" \
        : "=a"(retval) \
        : [number] "i"(NUMBER), [arg0] "r"(ARG0), [arg1] "r"(ARG1), \
          [arg2] "r"(ARG2), [arg3] "r"(ARG3) \
        : "memory"); \
    retval; \
})

```

인자가 4개인 경우에 대해서도 syscall 매크로를 선언했고

```

int fibonacci(int n) { return syscall1(SYS_FIBONACCI, n); }

int max_of_four_int(int a, int b, int c, int d) {
    return syscall4(SYS_MAX_OF_FOUR_INT, a, b, c, d);
}

```

앞서 언급한 시스템 콜을 호출하도록 구현했다. 이 함수들은 additional.c에서 호출했다.


```

/* System call numbers. */
enum
{
    /* Projects 2 and later. */
    SYS_HALT,           /* Halt the operating system. */
    SYS_EXIT,           /* Terminate this process. */
    SYS_EXEC,           /* Start another process. */
    SYS_WAIT,           /* Wait for a child process to die. */
    SYS_CREATE,         /* Create a file. */
    SYS_REMOVE,         /* Delete a file. */
    SYS_OPEN,           /* Open a file. */
    SYS_FILESIZE,       /* Obtain a file's size. */
    SYS_READ,           /* Read from a file. */
    SYS_WRITE,          /* Write to a file. */
    SYS_SEEK,           /* Change position in a file. */
    SYS_TELL,           /* Report current position in a file. */
    SYS_CLOSE,          /* Close a file. */
    SYS_FIBONACCI,      /* Return the nth Fibonacci number. */
    SYS_MAX_OF_FOUR_INT, /* Return the max of four integers. */

    /* Project 3 and optionally project 4. */
    SYS_MMAP,           /* Map a file into memory. */
    SYS_MUNMAP,         /* Remove a memory mapping. */

    /* Project 4 only. */
    SYS_CHDIR,          /* Change the current directory. */
    SYS_MKDIR,          /* Create a directory. */
    SYS_READDIR,        /* Reads a directory entry. */
    SYS_ISDIR,          /* Tests if a fd represents a directory. */
    SYS_INUMBER         /* Returns the inode number for a fd. */
};

#endif /* lib/syscall-nr.h */

```

pintos/src/lib/syscall-nr.h에 System Call Number도 등록한다.

Src/examples/additional.c 파일을 만들어

```

#include <stdio.h>
#include <syscall.h>

int atoi(const char *str);

int main(int argc, char *argv[]) {
    if (argc != 5) {
        printf("Usage: ./additional [num 1] [num 2] [num 3] [num 4]\n");
        return -1;
    }
    int a = atoi(argv[1]);
    int b = atoi(argv[2]);
    int c = atoi(argv[3]);
    int d = atoi(argv[4]);
    printf("%d %d\n", fibonacci(a), max_of_four_int(a, b, c, d));
    return 0;
}

int atoi(const char *str) {
    int res = 0; // Initialize result
    for (int i = 0; str[i] != '\0' && str[i] >= '0' && str[i] <= '9'; i++) {
        res = res * 10 + (str[i] - '0');
    }
    return res;
}

```

다음과 같이 입력을 읽어 atoi 함수를 구현해 정수화 시키고 첫번째 인자에 대해서만 피보나치 수, 모든 인자에 대해 최댓값을 구하도록 구현했다.

```
SRCDIR = ..

# Test programs to compile, and a list of sources for each.
# To add a new test, put its name on the PROGS list
# and then add a name_SRC line that lists its source files.
PROGS = cat cmp cp echo halt hex-dump ls mcat mcp mkdir pwd rm shell \
        bubblesort lineup matmult recursor additional

# Should work from project 2 onward.
cat_SRC = cat.c
cmp_SRC = cmp.c
cp_SRC = cp.c
echo_SRC = echo.c
halt_SRC = halt.c
hex-dump_SRC = hex-dump.c
lineup_SRC = lineup.c
ls_SRC = ls.c
recursor_SRC = recursor.c
rm_SRC = rm.c
additional_SRC = additional.c

# Should work in project 3; also in project 4 if VM is included.
bubblesort_SRC = bubblesort.c
matmult_SRC = matmult.c
mcat_SRC = mcat.c
mcp_SRC = mcp.c

# Should work in project 4.
mkdir_SRC = mkdir.c
pwd_SRC = pwd.c
shell_SRC = shell.c

include $(SRCDIR)/Make.config
include $(SRCDIR)/Makefile.userprog
```

해당 디렉토리의 makefile을 확인하면 additional.c도 컴파일되도록 수정하였다.

다음 장에

C. 시험 및 평가 내용

- fibonacci 및 max_of_four_int 시스템 콜 수행 결과를 캡처하여 첨부.

```
SeaBIOS (version 1.16.3-debian-1.16.3-2)
Booting from Hard Disk...
PPiiLLoo  hhddaa1
1
LLooaaddiinngg.....
Kernel command line: -f -q extract run 'additional 10 20 62 40'
Pintos booting with 3,968 kB RAM...
367 pages available in kernel pool.
367 pages available in user pool.
Calibrating timer... 665,190,400 loops/s.
ide0: unexpected interrupt
hda: 5,040 sectors (2 MB), model "QM00001", serial "QEMU HARDDISK"
hda1: 196 sectors (98 kB), Pintos OS kernel (20)
hda2: 4,096 sectors (2 MB), Pintos file system (21)
hda3: 100 sectors (50 kB), Pintos scratch (22)
ide1: unexpected interrupt
filesystem: using hda2
scratch: using hda3
Formatting file system...done.
Boot complete.
Extracting ustar archive from scratch device into file system...
Putting 'additional' into the file system...
Erasing ustar archive...
Executing 'additional 10 20 62 40':
55 62
additional: exit(0)
Execution of 'additional 10 20 62 40' complete.
Timer: 61 ticks
Thread: 2 idle ticks, 58 kernel ticks, 1 user ticks
hda2 (filesystem): 56 reads, 204 writes
hda3 (scratch): 99 reads, 2 writes
Console: 953 characters output
Keyboard: 0 keys pressed
Exception: 0 page faults
Powering off...
```

다음과 같이 10 20 62 40을 입력했을 때 잘 작동하는 것을 확인할 수 있다.

pass tests/userprog/args-none	pass tests/userprog/exec-once
pass tests/userprog/args-single	pass tests/userprog/exec-arg
pass tests/userprog/args-multiple	pass tests/userprog/exec-bound
pass tests/userprog/args-many	FAIL tests/userprog/exec-bound-2
pass tests/userprog/args-dbl-space	FAIL tests/userprog/exec-bound-3
pass tests/userprog/sc-bad-sp	pass tests/userprog/exec-multiple
pass tests/userprog/sc-bad-arg	pass tests/userprog/exec-missing
pass tests/userprog/sc-boundary	pass tests/userprog/exec-bad-ptr
pass tests/userprog/sc-boundary-2	pass tests/userprog/wait-simple
FAIL tests/userprog/sc-boundary-3	pass tests/userprog/wait-twice
pass tests/userprog/halt	pass tests/userprog/wait-killed
pass tests/userprog/exit	pass tests/userprog/wait-bad-pid
	pass tests/userprog/multi-recurse